Task 4: Order Management using Joins

Concepts Used: Joins, Subqueries, ORDER BY, CSS layout

Create tables for:

Customers

Orders

Products

Display a customer order history using JOIN queries and a subquery to find:

Highest value order

Most active customer

Real-Time Usage: E-commerce and retail systems.

Aim

To develop an Order Management System that displays customer order history using **JOIN queries**, and identifies the **highest value order** and **most active customer** using subqueries and aggregation.

Algorithm

1. Create tables for customers, products, and orders
2. Insert sample data into all tables
3. Connect PHP to the MySQL database
4. Use JOIN queries to combine customer, product, and order details
5. Calculate total order value using price and quantity
6. Use a subquery to find the highest value order
7. Use GROUP BY and COUNT to find the most active customer
8. Display all results on a webpage using HTML and CSS

Program:

```
CREATE DATABASE order_db;

USE order_db;
```

```sql
-- Customers Table
CREATE TABLE customers (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50)
);

-- Products Table
CREATE TABLE products (
    id INT AUTO_INCREMENT PRIMARY KEY,
    product_name VARCHAR(50),
    price INT
);

-- Orders Table
CREATE TABLE orders (
    id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT,
    product_id INT,
    quantity INT,
    order_date DATE
);
INSERT INTO customers (name) VALUES
('Preethi'),
('Ravi'),
('Anu'),
('Kiran');
```

```
INSERT INTO products (product_name, price) VALUES
('Laptop', 50000),
('Phone', 20000),
('Tablet', 15000),
('Headphones', 3000);
INSERT INTO orders (customer_id, product_id, quantity, order_date) VALUES
(1, 1, 1, '2024-01-10'),
(1, 2, 2, '2024-02-15'),
(2, 2, 1, '2024-03-01'),
(3, 3, 3, '2024-03-10'),
(4, 4, 5, '2024-03-12'),
(2, 1, 1, '2024-04-01');
SELECT c.name, p.product_name, p.price, o.quantity,
       (p.price * o.quantity) AS total, o.order_date
FROM orders o
JOIN customers c ON o.customer_id = c.id
JOIN products p ON o.product_id = p.id
ORDER BY o.order_date;
SELECT c.name, (p.price * o.quantity) AS total
FROM orders o
JOIN customers c ON o.customer_id = c.id
JOIN products p ON o.product_id = p.id
WHERE (p.price * o.quantity) = (
    SELECT MAX(p.price * o.quantity)
    FROM orders o
    JOIN products p ON o.product_id = p.id
);
```

```
SELECT c.name, COUNT(*) AS total_orders
FROM orders o
JOIN customers c ON o.customer_id = c.id
GROUP BY c.name
ORDER BY total_orders DESC
LIMIT 1;
```

Output:

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

name	product_name	price	quantity	total	order_date 1
Preethi	Laptop	50000	1	50000	2024-01-10
Preethi	Phone	20000	2	40000	2024-02-15
Ravi	Phone	20000	1	20000	2024-03-01
Anu	Tablet	15000	3	45000	2024-03-10
Kiran	Headphones	3000	5	15000	2024-03-12
Ravi	Laptop	50000	1	50000	2024-04-01

Show all | Number of rows: 25 Filter rows: Search this table

Query results operations

Print Copy to clipboard Export Display chart Create view

Console

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

name	total
Preethi	50000
Ravi	50000

Show all | Number of rows: 25 Filter rows: Search this table

Query results operations

Print Copy to clipboard Export Display chart Create view

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Extra options

name	total_orders
Preethi	2

Query results operations

Print Copy to clipboard Export Display chart Create view

Result

The order management system successfully retrieves customer order history using JOIN queries. It also identifies the highest value order and most active customer using subqueries and aggregation.